

1.1.1 ¿Qué es un módulo?

El código de computadora tiene una tendencia a crecer. Podemos decir que el código que no crece probablemente sea completamente inutilizable o esté abandonado. Un código real, deseado y ampliamente utilizado se desarrolla continuamente, ya que tanto las demandas de los usuarios como sus expectativas se desarrollan de manera diferente.

Un código que no puede responder a las necesidades de los usuarios se olvidará rápidamente y se reemplazará instantáneamente con un código nuevo, mejor y más flexible. Se debe estar preparado para esto, y nunca pienses que tus programas están terminados por completo. La finalización es un estado de transición y generalmente pasa rápidamente, después del primer informe de error. Python en sí es un buen ejemplo de cómo actúa esta regla.

El código creciente es, de hecho, un problema creciente. Un código más grande siempre significa un mantenimiento más difícil. La búsqueda de errores siempre es más fácil cuando el código es más pequeño (al igual que encontrar una rotura mecánica es más simple cuando la maquinaria es más simple y pequeña).

Además, cuando se espera que el código que se está creando sea realmente grande (puedes usar el número total de líneas de código como una medida útil, pero no muy precisa, del tamaño del código) entonces, se deseará, o más bien, habrá la necesidad de dividirlo en muchas partes, implementado en paralelo por unos cuantos, una docena, varias docenas o incluso varios cientos de desarrolladores.

Por supuesto, esto no se puede hacer usando un archivo fuente grande, el cual esta siendo editado por todos los programadores al mismo tiempo. Esto seguramente conducirá a un desastre.

Si se desea que dicho proyecto de software se complete con éxito, se deben tener los medios que permitan:

- Dividir todas las tareas entre los desarrolladores.
- Después, unir todas las partes creadas en un todo funcional.

Por ejemplo, un determinado proyecto se puede dividir en dos partes principales:

- La interfaz de usuario (la parte que se comunica con el usuario mediante widgets y una pantalla gráfica).
- La lógica (la parte que procesa los datos y produce resultados).

Cada una de estas partes se puede (muy probablemente) dividir en otras más pequeñas, y así sucesivamente. Tal proceso a menudo se denomina **descomposición**.

Por ejemplo, si te pidieran organizar una boda, no harías todo tú mismo: encontrarías una serie de profesionales y dividirías la tarea entre todos.

¿Cómo se divide una pieza de software en partes separadas pero cooperantes? Esta es la pregunta. Los **módulos** son la respuesta.

1.1.2 ¿Cómo hacer uso de un módulo?

Entonces, ¿qué es un módulo? El [Tutorial de Python](#) lo define como **un archivo que contiene definiciones y sentencias de Python**, que se pueden importar más tarde y utilizar cuando sea necesario.

El manejo de los módulos consta de dos cuestiones diferentes:

- El primero (probablemente el más común) ocurre cuando se desea utilizar un módulo ya existente - escrito por otra persona o creado por el programador mismo en algún proyecto complejo: en este caso, se

considera al programador como el **usuario** del módulo.

- El segundo ocurre cuando se desea crear un nuevo módulo, ya sea para uso propio o para facilitar la vida de otros programadores - aquí tu eres el **proveedor** del módulo.

Discutamos ambos por separado.

En primer lugar, un módulo se identifica por su **nombre**. Si se desea utilizar cualquier módulo, se necesita saber su nombre. Se entrega una cantidad (bastante grande) de módulos junto con Python. Se puede pensar en ellos como una especie de “equipamiento adicional de Python”.

usuario
emplea un
módulo existente



proveedor
crea un
módulo nuevo



Todos estos módulos, junto con las funciones integradas, forman la **Biblioteca Estándar de Python** - un tipo especial de biblioteca donde los módulos desempeñan el papel de libros (incluso podemos decir que las carpetas desempeñan el papel de estanterías). Si deseas ver la lista completa de todos los «volúmenes» recopilados en esa biblioteca, se puede encontrar aquí: <https://docs.python.org/3/library/index.html>.

Cada módulo consta de entidades (como un libro consta de capítulos). Estas entidades pueden ser funciones, variables, constantes, clases y objetos. Si se sabe como acceder a un módulo en particular, se puede utilizar cualquiera de las entidades que almacena.

**funciones
matemáticas**



math



**nombre
del módulo**

Comencemos la discusión con uno de los módulos más utilizados, el que lleva por nombre **math**. Su nombre habla por sí mismo - el módulo contiene una rica colección de entidades (no solo funciones) que permiten a un programador implementar efectivamente cálculos que exigen el uso de funciones matemáticas, como *sen()* o *log()*.

1.1.3 Importando un módulo

Para que un módulo sea utilizable, hay que **importarlo** (piensa en ello como sacar un libro del estante). La importación de un módulo se realiza mediante una instrucción llamada **import**. Nota: **import** es también una palabra clave reservada (con todas sus implicaciones).

Supongamos que deseas utilizar dos entidades proporcionadas por el módulo **math**:

- Un símbolo (constante) que representa un valor preciso (tan preciso como sea posible usando aritmética de punto flotante doble) de π (aunque usar una letra griega para nombrar una variable es totalmente posible en Python, el símbolo se llama **pi** - es una solución más conveniente, especialmente para esa parte del mundo que ni tiene ni va a usar un Teclado Griego).

- Una función llamada `sin()` (el equivalente informático de la función matemática *seno*).



Ambas entidades están disponibles a través del módulo `math` , pero la forma en que se pueden usar depende en gran medida de como se haya realizado la importación.

La forma más sencilla de importar un módulo en particular es usar la instrucción para importar de la siguiente manera:

```
import math
```

La cláusula contiene:

- La palabra reservada `import` ;
- El **nombre del módulo** que se va a importar.

La instrucción puede colocarse en cualquier parte del código, pero debe colocarse **antes del primer uso de cualquiera de las entidades del módulo**.

Si quiere (o tiene que) importar más de un módulo, puede hacerlo repitiendo la cláusula **import** (preferida):

```
import math  
import sys
```

o enumerando los módulos después de la palabra clave reservada **import** , como aquí:

```
import math, sys
```

La instrucción importa dos módulos, primero uno llamado **math** y luego un segundo llamado **sys** .

La lista de módulos puede ser arbitrariamente larga.

1.1.4 Namespace

Para continuar, debes familiarizarte con un término importante: **namespace**. No te preocupes, no entraremos en detalles - esta explicación será lo más breve posible.

Un **namespace** es un espacio (entendido en un contexto no físico) en el que existen algunos nombres y los nombres no entran en conflicto entre sí (es decir, no hay dos objetos diferentes con el mismo nombre). Podemos decir que cada grupo social es un namespace - el grupo tiende a nombrar a cada uno de sus miembros de una manera única (por ejemplo, los padres no darían a sus hijos el mismo nombre).



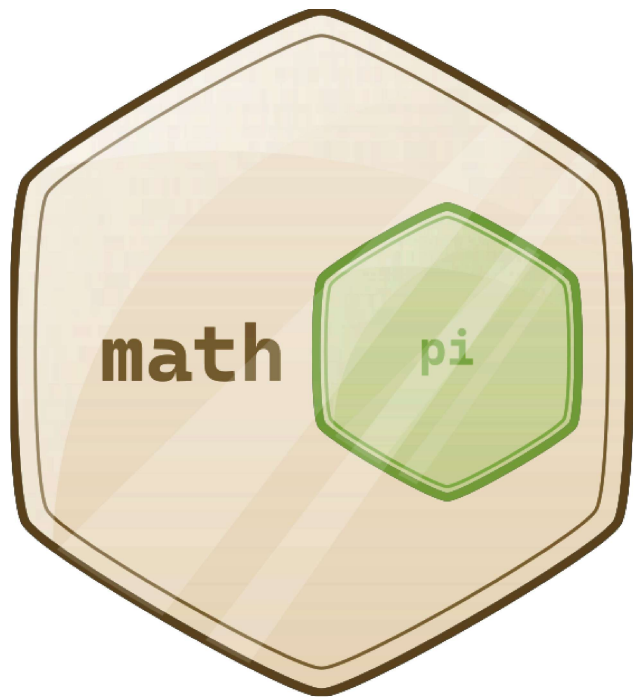
Esta singularidad se puede lograr de muchas maneras, por ejemplo, mediante el uso de apodos junto con los nombres (funcionará dentro de un grupo pequeño como una clase en una escuela) o asignando identificadores especiales a todos los miembros del grupo (el número de Seguro Social de EE. UU. es un buen ejemplo de tal práctica).

Dentro de un determinado namespace, cada nombre debe permanecer único.

Esto puede significar que algunos nombres pueden desaparecer cuando cualquier otra entidad de un nombre ya conocido ingresa al namespace. Mostraremos como funciona y como controlarlo, pero primero, volvamos a las importaciones.

Si el módulo de un nombre especificado **existe y es accesible** (un módulo es de hecho un **archivo fuente de Python**), Python importa su contenido, **se hacen conocidos todos los nombres definidos en el módulo**, pero no ingresan al namespace del código.

Esto significa que puedes tener tus propias entidades llamadas **sin** o **pi** y no serán afectadas en alguna manera por la importación.



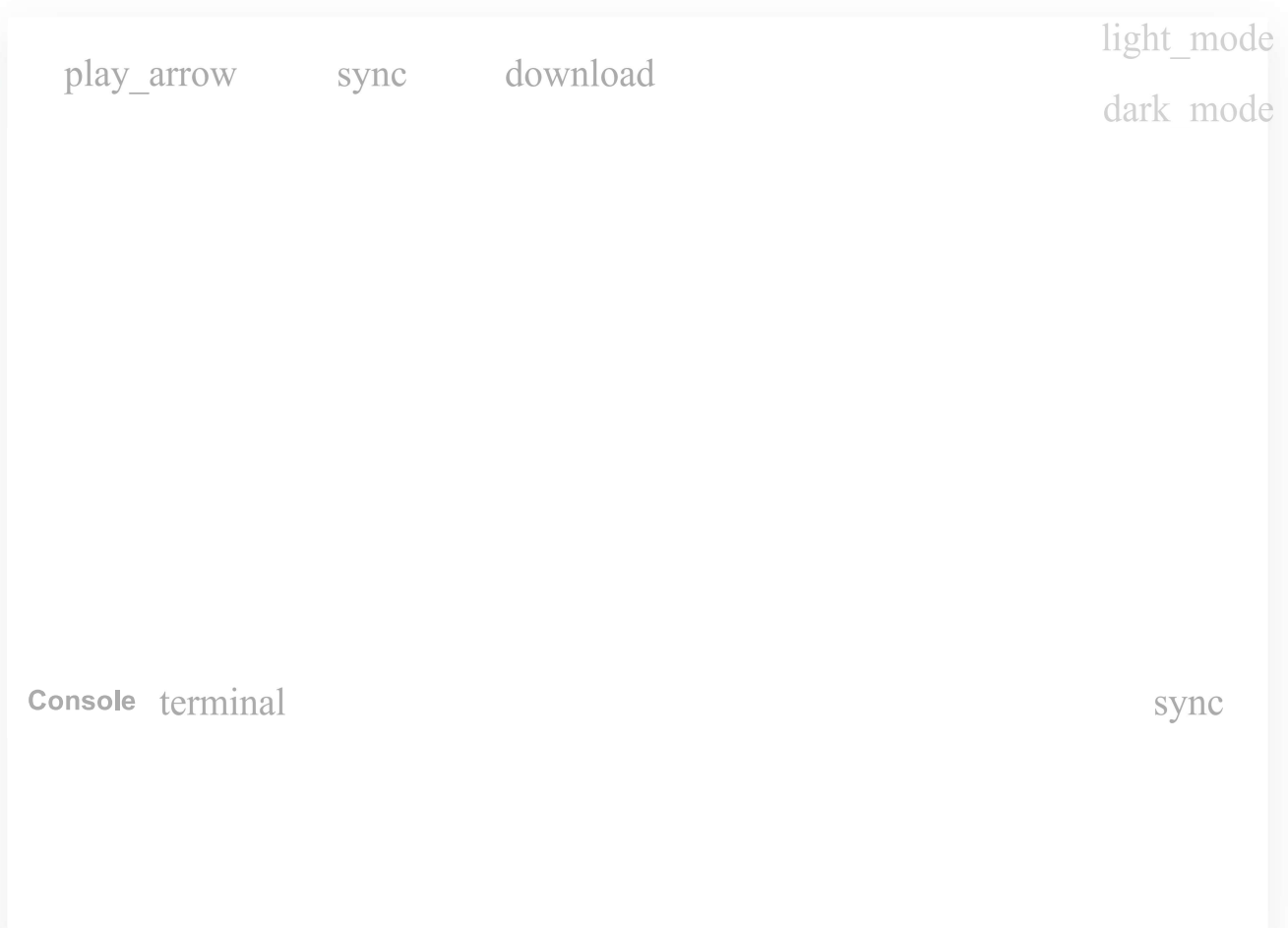
En este punto, es posible que te estes preguntando como acceder al **pi** el cual viene del módulo **math** .

Para hacer esto, se debe de mandar llamar el **pi** con el nombre del módulo original.

1.1.5 Importando un módulo: continuación

Este primer ejemplo no será muy avanzado - solo se desea imprimir el valor de **$\sin(\frac{1}{2}\pi)$** .

Observa el código en el editor. Así es como se prueba.



El código genera el valor esperado: **1.0** .

Observa el fragmento a continuación, esta es la forma en que se habilitan los nombres de `pi` y `sin` con el nombre de su módulo de origen:

```
math.pi  
math.sin
```

Es sencillo, se coloca:

- El **nombre del módulo** (`math`).
- Un **punto** (p.ej., `.`).
- El **nombre de la entidad** (`pi`).

Tal forma indica claramente el namespace en el que existe el nombre.

Nota: el uso de esto es **obligatorio** si un módulo ha sido importado con la instrucción `import` . No importa si alguno de los nombres del código y del namespace del módulo están en conflicto o no.

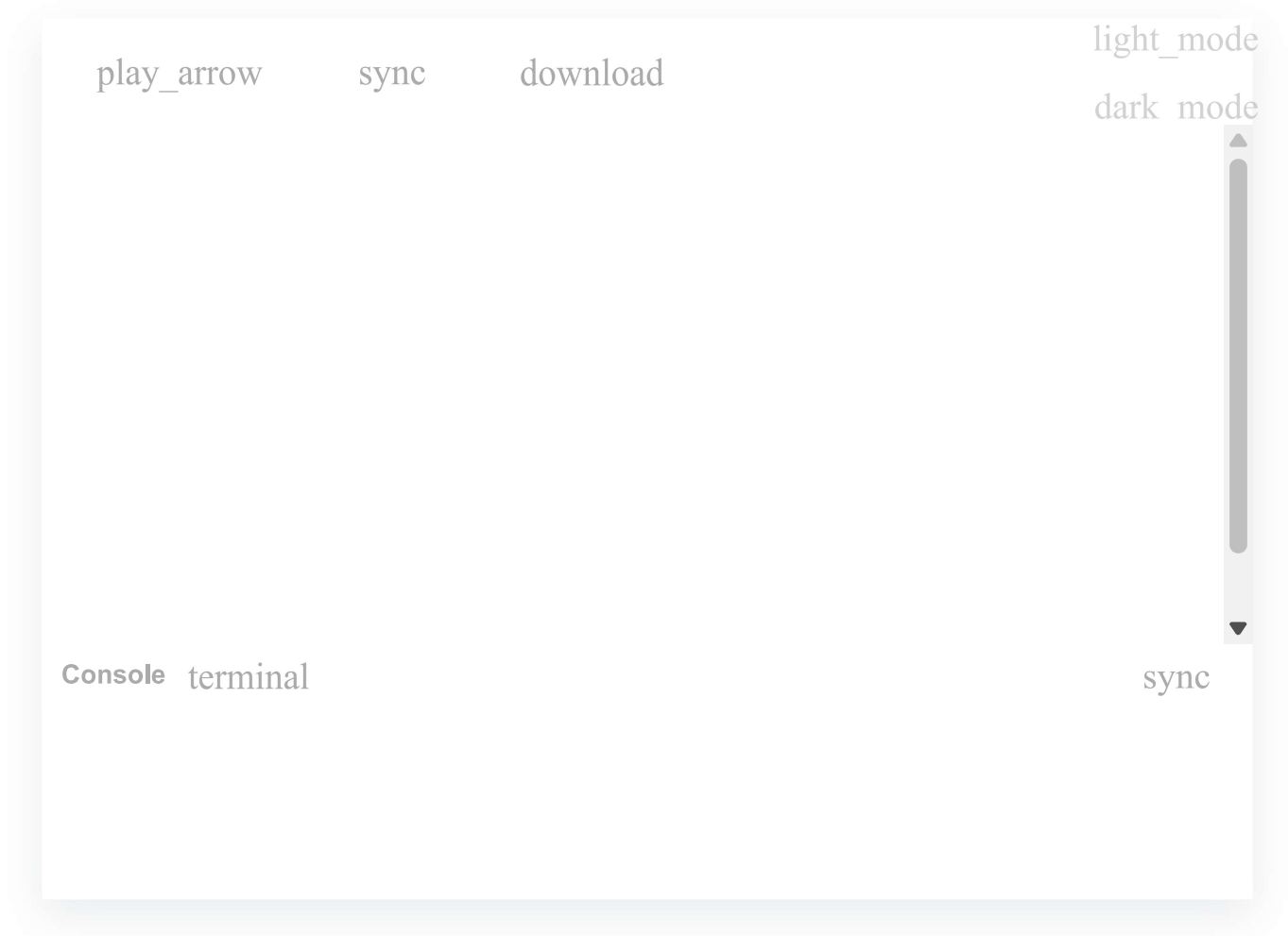
Nota: el eliminar cualquiera de las dos indicaciones del nombre del módulo hará que el código sea erróneo. No hay otra forma de entrar al namespace de `math` si se hizo lo siguiente:

```
import math
```

Ahora te mostraremos cómo pueden dos namespaces (el tuyo y el del módulo) coexistir.

Echa un vistazo al ejemplo en la ventana del editor.

Hemos definido nuestros propios **pi** y **sin** aquí.



Ejecuta el programa. El código debe producir la siguiente salida:

Output

0.99999999

1.0

Como puedes ver, las entidades no se afectan entre sí.

En el segundo método, la sintaxis del **import** señala con precisión que entidad (o entidades) del módulo son aceptables en el código:

```
from math import pi
```

La instrucción consta de los siguientes elementos:

- La palabra clave reservada **from** .
- El **nombre del módulo** a ser (selectivamente) importado.
- La palabra clave reservada **import** .
- El **nombre o lista de nombres de la entidad o entidades** las cuales estan siendo importadas al namespace.

La instrucción tiene este efecto:

- Las entidades listadas son las únicas que son **importadas del módulo indicado**.
- Los nombres de las entidades importadas pueden **ser accedidas dentro del código sin especificar el nombre del módulo de origen**.

Nota: no se importan otras entidades, únicamente las especificadas. Además, no se pueden importar entidades adicionales utilizando - una línea como esta:

```
print(math.e)
```

Esto ocasionará un error, (**e** es la constante de Euler: 2.71828...)

Reescribamos el código anterior para incorporar esta nueva técnica.

Aquí está:

play_arrow

sync

download

light_mode

dark mode

El resultado debe de ser el mismo que el anterior, se han empleado las mismas entidades: **1.0** . Copia y pega el código en el editor, y ejecuta el programa.

¿El código parece más simple? Quizás, pero el aspecto no es el único efecto de este tipo de importación. Veamos más a detalle esto.

Observa el código en el editor. Analízalo cuidadosamente:

- La línea 01: lleva a cabo la importación selectiva.
- La línea 03: hace uso de las entidades importadas y obtiene el resultado esperado (**1.0**).
- Las líneas 05 a la 12: redefinen el significado de **pi** y **sin** - en efecto, reemplazan las definiciones originales (importadas) dentro del namespace del código.
- La línea 15: retorna **0.99999999** , lo cual confirma nuestras conclusiones.

play_arrow

sync

download

light_mode

dark mode

Console terminal

sync

Hagamos otra prueba. Observa el código a continuación:

play_arrow

sync

download

light_mode

dark mode

Console terminal

sync

Aquí, se ha invertido la secuencia de las operaciones del código:

- Las líneas del 01 al 08: definen nuestro propio **pi** y **sin**.
- La línea 11: hace uso de ellas (0.99999999 aparece en pantalla).
- La línea 13: lleva a cabo la importación - los símbolos importados **reemplazan sus definiciones anteriores dentro del namespace**.
- La línea 15: retorna **1.0** como resultado.

1.1.6 Importando un módulo: *

En el tercer método, la sintaxis del `import` es una forma más agresiva que la presentada anteriormente:

```
from module import *
```

Como puedes ver, el nombre de una entidad (o la lista de nombres de entidades) se reemplaza con un solo asterisco (`*`).

Tal instrucción **importa todas las entidades del módulo indicado**.

¿Es conveniente? Sí, lo es, ya que libera del deber de enumerar todos los nombres que se necesiten.

¿Es inseguro? Sí, a menos que conozcas todos los nombres proporcionados por el módulo, **es posible que no puedas evitar conflictos de nombres**. Trata esto como una solución temporal e intenta no usarlo en un código regular.

1.1.7 La palabra clave `as`

Si se importa un módulo y no se esta conforme con el nombre del módulo en particular (por ejemplo, si es el mismo que el de una de sus entidades ya definidas) puede dársele otro nombre: esto se llama **aliasing** o renombrado.

Aliasing (renombrado) hace que el módulo se identifique con un nombre diferente al original. Esto también puede acortar los nombres originales.

La creación de un alias se realiza junto con la importación del módulo, y exige la siguiente forma de la instrucción `import`:

```
import module as alias
```

El «module» identifica el nombre del módulo original mientras que el «alias» es el nombre que se desea usar en lugar del original.

Nota: **as** es una palabra clave reservada.

1.1.8 Aliasing (renombrado)

Si necesitas cambiar la palabra **math** , puedes introducir tu propio nombre, como en el ejemplo:

```
import math as m
```

```
print(m.sin(m.pi/2))
```

Nota: después de la ejecución exitosa de una importación con alias, **el nombre original del módulo se vuelve inaccesible** y no debe ser utilizado.

A su vez, cuando usa la variante **from module import name** y se necesita cambiar el nombre de la entidad, se crea un alias para la entidad. Esto hará que el nombre sea reemplazado por el alias que se elija.

Así es como se puede hacer:

```
from module import name as alias
```

Como anteriormente, el nombre original (sin alias) se vuelve inaccesible.

La frase **name as alias** puede repetirse: puedes emplear comas para separar las frases, como a continuación:

```
from module import n as a, m as b, o as c
```

El ejemplo puede parecer un poco extraño, pero funciona:

```
from math import pi as PI, sin as sine  
  
print(sine(PI/2))
```

Ahora estás familiarizado con los conceptos básicos del uso de módulos. Permítenos mostrarte algunos módulos y algunas de sus entidades útiles.

1.1.9 RESUMEN DE LA SECCIÓN

1. Si deseas importar un módulo como un todo, puedes hacerlo usando la sentencia `import nombre_del_módulo`. Puedes importar más de un módulo a la vez utilizando una lista separada por comas. Por ejemplo:

```
import mod1
import mod2, mod3, mod4
```

Aunque la última forma no se recomienda por razones estilísticas, y es mejor y más bonito expresar la misma intención de una forma más detallada y explícita, como por ejemplo:

```
import mod2
import mod3
import mod4
```

2. Si un módulo se importa de la manera anterior y desea acceder a cualquiera de sus entidades, debes anteponer el nombre de la entidad empleando la **notación con punto**. Por ejemplo:

```
import my_module

result = my_module.my_function(my_module.my_data)
```

El fragmento de código utiliza dos entidades que provienen del módulo **my_module** : una función llamada **my_function()** y una variable con el nombre **my_data** . Ambos nombres **deben tener el prefijo my_module** . Ninguno de los nombres de entidad importados entra en conflicto con los nombres idénticos existentes en el namespace de tu código.

3. Se te permite no solo importar un módulo como un todo, sino también importar solo entidades individuales de él. En este caso, las entidades importadas **no deben especificar** el prefijo cuando son empleadas. Por ejemplo:

```
from module import my_function, my_data

result = my_function(my_data)
```

La forma anterior, a pesar de su atractivo, no se recomienda debido al peligro de causar conflictos con los nombres derivados de la importación del namespace del código.

4. La forma más general de la sentencia anterior te permite importar **todas las entidades** ofrecidas por un módulo:

```
from my_module import *

result = my_function(my_data)
```

Nota: la variante de esta importación no se recomienda debido a las mismas razones que antes (la amenaza de un conflicto de nombres es aún más peligrosa aquí).

5. Puedes cambiar el nombre de la entidad importada "sobre la marcha" utilizando la frase `as` del `import` . Por ejemplo:

```
from module import my_function as fun, my_data as dat

result = fun(dat)
```

1.1.10 CUESTIONARIO DE SECCIÓN

Pregunta 1: Quieres invocar la función `make_money()` contenida en el módulo llamado `mint` . Tu código comienza con la siguiente línea:

```
import mint
```

¿Cuál es la forma adecuada de invocar a la función?

Revisar

Pregunta 2: Deseas invocar la función `make_money()` contenida en el módulo llamado `mint` . Tu código comienza con la siguiente línea:

```
from mint import make_money
```

¿Cuál es la forma adecuada de invocar a la función?

Revisar

Pregunta 3: Has escrito una función llamada `make_money` por tu cuenta. Necesitas importar una función con el mismo nombre del módulo `mint` y no deseas cambiar el nombre de ninguno de tus nombres previamente definidos. ¿Qué variante de la sentencia `import` puede ayudarte con el problema?

Revisar

Pregunta 4: ¿Qué forma de invocación de la función `make_money` es válida si tu código comienza con la siguiente línea?

```
from mint import *
```

Revisar